

Self-Healing 5G Timing Security

A plain-English walkthrough of the whole project — every part, the data, and the results

Project: AI-Native Self-Healing O-RAN Network using a Digital Twin
Prepared for the team — no telecom background needed

Item	Status
Software	Complete — 56 automated tests passing
Version	Commit 4e4bd16 · restore point known-good-2026-08-07
Real-data result	~99.6–99.96% protection on real attacks; 2.37% false alarms
Hardware (Tier-3)	Not started — the next phase
Note	Most build work was AI-generated under human direction

1 · The project in 30 seconds

A 5G base station is split into pieces that talk over a link called the **fronthaul**. Those pieces must agree on **time** to about **100 nanoseconds** (a ten-millionth of a second). If the timing breaks, phone service fails in roughly **2 seconds**.

The clock is shared using a protocol called **PTP**, which has no built-in security. So a faulty or malicious clock can knock a base station offline. The hard part: a harmless fault (lost GPS, network congestion) and a deliberate attack look **almost identical**.

What we built: software that watches the timing, decides whether a problem is a **harmless fault** or a **real attack**, tests a fix in a fast simulation (a 'digital twin'), and applies it — all inside the 2-second deadline. And if it can't trust its own data, it refuses to say 'healthy'.

Where this sits in the bigger picture

- **Field:** 5G / O-RAN network security.
- **Zoom in:** the Open Fronthaul link between the radio and the baseband.
- **Zoom in again:** the **S-plane** — the traffic that carries the clock. That is our topic.

2 · The core problem: fault vs attack

When timing drifts, the symptom is the same whatever the cause. The correct response, though, is **opposite** — so telling them apart is the whole game.

Harmless fault (H0)	Malicious attack (H1)
GPS signal lost — clock rides its internal backup	GPS spoofed — clock believes a fake time
Network congestion adds delay/jitter	Attacker injects one-sided delay on purpose
A link fails and quality drops	Attacker forges 'I am the best clock' messages
A planned, legitimate switch-over	A rogue device impersonating the master

For a fault, the right move is usually to **wait / ride it out** — the real clock returns. For an attack, the right move is to **isolate the bad source and switch away**. Do the wrong one and you make the outage worse.

The project's central claim: a system that only shouts 'something is wrong' leaves the operator with a coin-flip under a 2-second deadline. Telling H0 from H1 is what makes an automatic response safe enough to attempt at all.

3 · How it works — the pipeline

Data flows through six stages. Each is a small, testable piece of software.

- 1 **Ingest** — read the timing data (from our simulator, a real capture file, or a live PTP program) into one common format.
- 2 **Features** — cut the stream into 0.4-second windows and compute **28 numbers** per window (timing, message patterns, clock-election behaviour, oscillator health).
- 3 **Discriminator** — a Random Forest labels each window: benign fault (H0) or attack (H1).
- 4 **Open-set safety net** — anything unlike anything seen before is flagged **UNKNOWN** (so a brand-new attack isn't forced into a wrong box). A 2-of-3 vote ignores single noisy windows.
- 5 **Digital twin** — before acting, forecast what each candidate fix would do, and rate its own confidence.
- 6 **Healing loop** — pick and commit the safest fix within 1 second. If the data can't be trusted, refuse and fall back to a safe default.

The 28 features, in plain groups: timing (offset, delay, jitter) · message regularity · message rate · clock-election (BMCA) behaviour · time-source (GPS/SyncE) status · oscillator consistency. Grouping them is what lets the system catch attacks it was never explicitly taught.

4 · The data — where the numbers come from

This is the question people ask most, so it's worth being exact. There are **two** sources, and they carry very different weight.

Source 1 — our simulator (synthetic)

Physics-based software that fakes a realistic PTP clock and lets us stamp in perfectly-labelled faults and attacks. It's our **sandbox** — great for building and debugging. Most of the big-looking simulator percentages come from here, so they're the **weakest** evidence.

Source 2 — real TIMESAFE captures (the real evidence)

Actual PTP recordings from a real 5G testbed at Northeastern University, published with their research. Five independent sessions of Announce and Sync attacks, each with a benign baseline. **When we claim ~99% on real attacks, it's on these.**

Real session	Type	Rows
announce_session_1 / 2 / 3	Real Announce attacks + benign	~4.5k–47k each
sync_followup_session	Real Sync/Follow-Up attack + benign	~8.4k
sync_singlestep_session	Real single-step attack + benign	~9k
Live netem runs (ours)	Real ptp4l over a virtual link	5,665 windows

Honest limit: only five independent real sessions, and no recording of a benign *planned* master change — the one confounder we'd most like to have.

5 · Results

On the simulator (sandbox — 8 random seeds)

Metric	Value	Meaning
Accuracy	0.991 ± 0.002	Benign-vs-attack called correctly
Recovery success	1.000	The loop fixed every scenario
Wrong-action rate	0.000	Never applied the opposite fix
Time to recovery	0.933 s	Inside the 2-second deadline

On REAL data (the evidence that counts)

Test	Result	Reading
Real Announce/BMCA attack	99.96%	protection
Real Sync/Follow-Up attack	99.66%	protection
Real single-step attack	99.66%	protection
Benign false alarms	4.49% → 2.37%	cut by the 2-of-3 vote
Unseen healthy-looking GNSS spoof	0.00%	the known hard limit

Live (real ptp4l running)

- **5,665** real-trace windows captured across four impairment scenarios.
- Decision latency: mean **0.081 s**, worst **0.404 s** — far inside 2 s.
- After the master clock was killed: **0** windows wrongly called 'healthy'.

6 · The one real bug we found — and fixed

Live testing exposed a genuine safety flaw. Under heavy packet loss, the timing tool returned nothing, our software substituted a **zero**, and a zero looks exactly like **perfect** synchronisation. So the system reported 'healthy' at the very moment it had gone blind — and an attacker could trigger this on purpose by flooding the link.

The fix: treat missing data as its own state, never as a number. The software now tracks whether each measurement was really observed, and refuses to call anything 'healthy' unless it can prove it saw the data. This is called **failing closed**. Ten tests, written before the fix, lock it in.

We then proved it on **real** PTP software: when we killed the master clock, the tool kept confidently reporting its **last known numbers** forever — worse than a zero. Our software reads the connection state instead of trusting the numbers, so it caught both versions.

Honest limitations

- No real hardware yet (no PTP timing card, GPS receiver, or radio unit tested).
- Only five independent real attack sessions.
- A perfectly healthy-looking GPS spoof can't be caught from a single clock — a known theoretical limit, not a coding bug.
- These are research measurements, not telecom certification.

7 · What's done, and what's next

Area	Status
Software build	Complete. 56 tests passing, one command reproduces everything.
Software validation	Complete on real recorded data and live real ptp4l.
The safety bug	Found, fixed, hardened, and verified on real software.
Hardware (Tier-3)	Not started — real timing card, GPS grandmaster, authenticated GPS.
Research novelty add-on	Designed: an 'evasion attack' (software-only) to strengthen the paper.

The next step in one line

Build an **evasion attack**: instead of only defending, act as the attacker and design a timing attack that slips past the detector while still breaking the clock. It needs no hardware, reuses what we already built, and turns the story into 'new attack → our defence stops it' — which is what strong publications want. This is being built separately.

One-sentence summary for the slide: we built and validated, on real 5G timing data, a system that tells timing faults from attacks and heals inside the 2-second deadline — failing safely when it can't trust its data — with the hardware validation and an evasion-attack study as the clear next steps.

8 · Mini-glossary

Term	Meaning
O-RAN	Open, multi-vendor way of building 5G base stations.
Fronthaul	The link between the radio unit and the baseband unit.
S-plane	The fronthaul traffic that carries the clock. Our topic.
PTP (IEEE 1588)	The protocol that shares time across the network. No built-in security.
GNSS / GPS	Satellite time source. Can be jammed or spoofed.
Holdover	Running on an internal backup clock after losing the reference. Benign.
BMCA	The 'best clock wins' election attackers exploit by forging attributes.
H0 / H1	Our labels: H0 = benign fault, H1 = malicious attack.
Digital twin	A fast model that predicts what a fix will do before we apply it.
Fail-closed	Refusing to certify 'healthy' when the data can't be trusted.
Open-set / UNKNOWN	Being able to answer 'none of the above' for a brand-new attack.
Evasion attack	An attack shaped to slip past the detector — the planned next study.